

NuWro design

Software and probabilistic architecture

Cezary Juszczak

September 18, 2026

Main assumptions

- Made for **physicists** *nonprogrammers*
- Simplified creation of new interaction channels
- Easy to implement formulae from publications
 - System of units: $c = \hbar = MeV = 1$.
 - Implemented vector and 4vector operators, scalar products, powers,...
- Cross sections calculated on the go (not from tables)
- Input files: **text based**, easy to edit and understand.
- Output file: in **.root** format, easy to analyze, current standard.
- Architecture centered around the `event` structure
- Very limited external dependencies (only Phythia 6 and **root**)

System of Units

$$c = \hbar = \text{MeV} = 1$$

- This choice makes it easy to input theoretical models.
- Also makes many formulae shorter $\Rightarrow$ more readable and faster to calculate
- Usage examples:

```
double m_p= 903*MeV;  
cout<< "m_p = " << m_e/kg << "kg\n";
```

In house implementation of vectors and particles

- The basic objects like 3D and 4D vectors (`vec` and `vect` classes respectively) with addition, scalar and vector products, boosts and rotations and many useful methods are implemented.
- The `particle` is a subclass of `vect` so you can add particles to obtain total 4 momentum or subtract them to obtain 4 momentum transfer.
- The velocity of particle `p` is calculated in the parent `vect` class as `p.v()`. So the CMS frame of particles `p` and `q` moves with velocity `vec v = (p + q).v();`.
- `particle` fields include `mass`, `pdg` code, four-position `r` (useful for tracking inside nucleus during reinteractions) and some others used for event analysis.
- The `spin()` and `charge()` are calculated from the `pdg` code.
- The function `decay(p+k, &k1, &p1);` can be used to create simple kinematics isotropic in CMS. Only masses of particles `k1`, and `p1` need to be preset.

Some programmatic decisions - parameters

- All parameters used by the program combined in one object `params` .
- Each parameter is defined using macro `PARAM(type, name, default_value)\` in `params_all.h` with type limited to: `bool` , `int` , `double` , `vec` , `string` , and `line` . This macro is redefined three times for constructing, reading and writing `params` object.
- Every configurable object created in NuWro is constructed based on `params` .
- A copy of `params` is included in every generated event.
- The values of some parameter can change from event to event (notably `nucleus_p` and `nucleus_n`) - a remnant from times where only single isotope targets were allowed.
- Decision not to depend on external dependencies saved many MB of source code.

Input file format

Just four kinds of lines are used:

```
parameter = value    // set the value or first line of the parameter
parameter += line    // append line to the multiline parameter string
# comments           // are used just for readability or showing possible parameter values
@file.txt            // include content of file.txt here - allows for using predefined configs
```

Command line parameters

Parameters are read from the file given on the command line as in `nuwro -i myparams.txt`. If it is not given, then file `params.txt` located in the working directory is used. If no such file exists, then the `data/params.txt` from the `nuwro` directory is read.

Any parameter can be altered in the command line when prefixed with `-p` switch:

```
nuwro -p energy = "1000 2000" -p pdg = 12
```

In fact the CL parameters behave like lines appended to the `params.txt` file. No special parsing code is needed. This solution is robust, predictable and facilitates scripting.

Beam types

- single flavour, uniform direction, with energy profile (type=0)
- mixed flavour, uniform direction, different energy profiles (type=1)
- file based (files generated by beam MC software) (type=2)

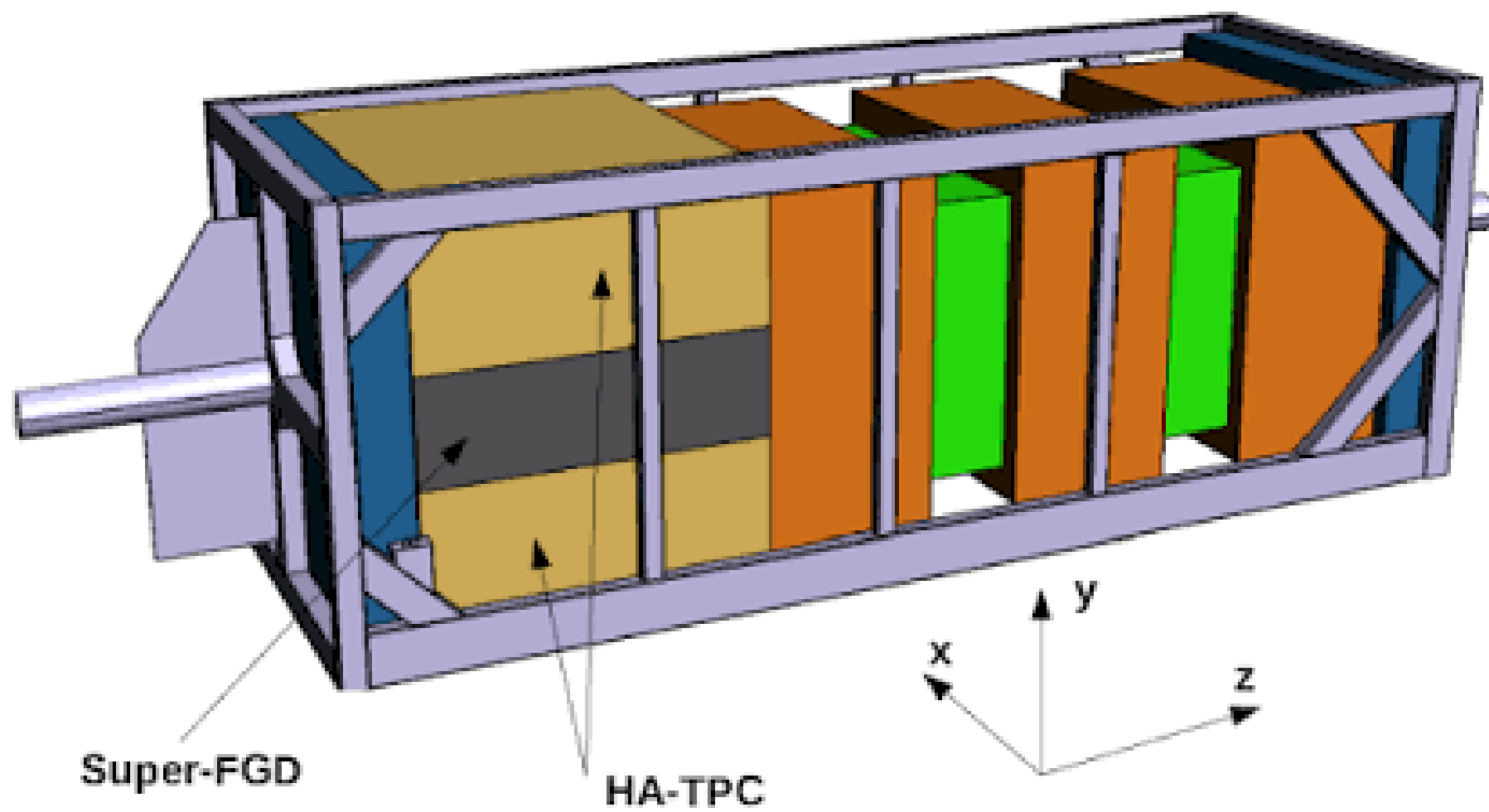
Every beam implementation must be a subclass of:

```
class beam {  
public:  
    virtual particle shoot(bool dis=0) = 0; // must be implemented  
    virtual double nu_per_POT(){ return 0;} // for use with detector geometries only  
    virtual ~beam(){}  
};
```

The method `beam::shoot(false)` returns neutrino according to the beam energy profile.

The method `beam::shoot(true)` returns neutrino from energy profile multiplied by energy.

ND280 detector



The ND280 detector beam and target

One of the finest achievements of the framework is the ability to accurately and very efficiently simulate events in arbitrarily complicated real world detector and beam defined by the GEANT4 detector simulation software.

The idea

- The detector geometry fits in the nuwro scheme similarly as some weighted mixture of isotopes. In the case of a mixture the target isotope is chosen according to its mass share in the mixture and sets `nucleus_p` , `nucleus_n` parameter values accordingly.
- In the case of detector geometry and realistic flux the final effect is the same: just one nucleus from all materials of the detector is picked up and put in the event structure together with its space coordinates.
- However, the process of choosing a nucleus from material is preceded by choosing a point in the detector geometry which contains this material.

The Detector Beam

- The beam is stored in a folder with **.root** files.
- Each file contains list of weighted neutrinos coming from beam simulation and corresponds to 10^6 POT (protons on target).
- Neutrino information given: pdg, position, momentum, energy and weight.
- Beam object loads all neutrinos from files to a vector of vectors (avoid too big memory chunk) and builds array of their cumulated weights.
- The `shoot` function searches this array for random percentage of the total weight in $\log(n)$ time using bisection and returns corresponding neutrino.
- Use this beam with the ND280 detector target as follows:

```
@beam/ND280.txt  
@target/ND280.txt
```

Detector Geometry with near detector beam (type = 2)

- The `.root` file contains the geometry description (from Geant4).
- `GetPoint(x,y,z)` returns the material (density, isotopic composition) at (x,y,z).
- Due to tiny neutrino cross section, no need to do the tracking.

Instead:

- For every neutrino from the beam:
 - accept it based on the **length** of its path inside region of interest (first rejection)
 - accept material from random point on the path based its **density** (second rejection)
 - in each case of rejection start over with a new neutrino.
- Choose isotope from the accepted material according its mass share.
- Use accepted neutrino and chosen isotope to generate event.
- All neutrinos (even rejected) are accounted for in the POT calculation.
- **NuWro events can be fed into the GEANT4 detector simulation.**

Detector geometry with beams of type 0 or 1 (uniform in space)

If beam is uniform in space choosing point of interaction is independent from the beam so the algorithm is even simpler:

- Select random point (x,y,z) from the region of interest.
- Find material and density at this point using `GetPoint(x, y, z)`
- Accept the point based on material density. In case of rejection start over.
- Get random isotope from the material with probability reflecting to its mass share.
- Generate event using the a neutrino from the beam and the chosen nucleus.

How to use ND280 detector geometry

- Just put in your `paramt.txt` line `@target/ND280.txt`
- Contents of `target/ND280.txt`

```
target_type=2
geo_file=target/ND280.root
geo_name=ND280Geometry
geo_o = 0 0 0 // coordinates of the center of the box: ox oy oz
geo_d = 1500 1500 3200 // half dimension of the box: dx dy dz
nucleus_target=2
geom_length_units=mm

# converts density used in geometry file into g/cm3
geom_density_convert=6.24151e+18
```

- Similar files can be prepared for other detectors. You need geometry file from GEANT4 simulation, name of the master volume, and then define your box of interest. The generated events will be limited to that box.

The output format

Output is given as a root file containing a TTree object with just one branch named `e` which contained where every `e` is just an `event`. To be discussed later.

Thanks to this approach the initial `e` may be omitted and particular event details can be easily accessed.

```
treeout->Draw("out.mass()"); //masses of outgoing partiles.  
treeout->Draw("n()"); // number of outgoing partiles  
treeout->Draw("nof(111)","flag.dis*flag.cc"); // number of  $\pi^0$  produred in DIS CC events  
treeout->Draw("out.momentum()", "out.pdg==111");// momenta of outgoing  $\pi^0$   
treeout->Draw("all.r.x:all.r.y:all.r.z");// places where the interations took place FSI code  
treeout->Draw("q2()", "flag.dis"); // differential cross section in the DIS channel
```

The event structure

```
class event:public TObject
{
    public:
        flags flag;           ///< flags for convenient filtering of the events in root scripts..
        params par;           ///< copy of all the input parameters.
                               ///< NOTE: some parameters (e.g. nucleus_p, nucleus_n) can vary.
                               ///< from event to event (e.g. in case of the detector simulation)

        vector < particle > in;   ///< vector of incoming particles
        vector < particle > temp; ///< vector of temporary particles (daughters of primary vertex in DIS)
        vector < particle > out;  ///< vector of outgoing particles (before fsi)
        vector < particle > post; ///< vector of particles leaving the nucleus
        vector < particle > all;  ///< vector of all particles (including temporary fsi particles)

        double weight;          ///< cross section of current event in cm^2 (set to total cross section on saving to file)
        double norm;            ///< norm of the initial neutrino (for weighted beams; not used).
        vec r;                  ///< position of the event inside the detector
        double density;         ///< density of the detector matter in point of the interaction
        int dyn;                ///< dynamics channel of the primary vertex. Possible values are:
        int nod[18];            ///< number of rescattering interactions of given type (see code)
        int pr;                 ///< number of protons in the residual nucleus
        int nr;                 ///< number of neutrons in the residual nucleus
        double r_distance;      ///< distance from nucleus center of absorption point (if happened)

        double res_jacobian;    ///< store Jacobian calculated in RESonant scattering for random kinematics
        double res_angrew;      ///< store xsec factor coming from angular distribution (for Delta)
        particle res_nu;        ///< store neutrino for reweighting
        vect res_q;             ///< store q for reweighting

    /// methods
    ....
};
```

Scheme of a single event generation

- take neutrino from the beam and put it in the `event.in` vector.
- use target definition to create target nucleus and store it in `nucleus` object.
- fill `params.nucleus_p` and `params.nucleus_n` accordingly.
- choose the dynamical channel for the primary vertex.
- ask `nucleus` to put initial nucleon in the `event.in` vector,
- call the corresponding routine e.g. `qeLevent(params, event, nucleus)` ;
 - the routine is obliged to: generate kinematics,
 - put outgoing particles to `event.out` vector,
 - calculate differential cross section and store it in the `event.weight`
 - in case of DIS: secondary particles coming from hadronization go to `event.tmp`
- run intranuclear cascade code, particles leaving nucleus put in the `event.post` vector.
 - if requested in `params` , store intermediate FSI particles in `event.all` vector.

Creating a toy model dynamical channel

Problem: write event generation procedure for elastic neutrino nucleon scattering isotropic in cms with total cross section 10^{-50}fm

```
double toyevent(params&p, event & e, nucleus &t)
{
    particle nu=e.in[0]; // beam neutrino
    particle N0=e.in[1]; // target nucleon
    particle nu1=nu;      // scattered neutrino
    particle N1=N0;       // nucleon with recoil

    decay(nu+N0, nu1, N1); // isotropic scattering in cms
    e.weight = 1e-50*fm;

    e.out.push_back(nu1);
    e.out.push_back(N1);

    if(t.pauli_blocking(N1))
        e.weight = 0;
}
```

Nuwro needs two runs to produce unweighted events

- Test run:
 - determine the total cross section for each channel (no FSI necessary)
 - produces equal numbers of weighted events in every enabled dynamic channel.
 - The average weight = the total cross section per nucleon (for this particular beam)
 - 100% efficiency $\rightarrow$ every event is used in calculation and can be saved to file.
- Production run:
 - Produces unweighted events (every weight = total cross section per nucleon)
 - Separately produces events precalculated number of events for each channel
 - The number of events in each channel proportional to the total cross sections
 - Include FSI reinteractions
- For determining the differential cross sections the test run is usually enough

Efficiency considerations and some problems solved

Is NuWro fast enough?

Efficiency

- General NuWro principle:
the average weight in a channel = the cross section in that channel.
- Test events (weighttd):

$$\text{Efficiency} = 100\%$$

- Real events:

$$\text{Efficiency} = \text{acceptance probability} = \frac{\text{average weight}}{\text{maximum weight}} \times 100\%.$$

DIS, RES problem

- For beams with high-energy tails the maximum weight in the DIS channel can exceed the average weight by a factor of 100 or more (Efficiency < 1%)

Solution of the DIS, RES problem

For **DIS** events cross section grows $\sim$ linearly with neutrino energy. For beams with high energy tails this leads to very poor efficiency.

Idea:

- Sample neutrinos according to probabilities **multiplied by neutrino energy**,
- Accept events according to the weight **divided by neutrino energy**.

Result:

- roughly **10 times higher efficiency** for DIS and RES for beams with energetic tails – in particular the **T2K beam on the ND280 detector**.

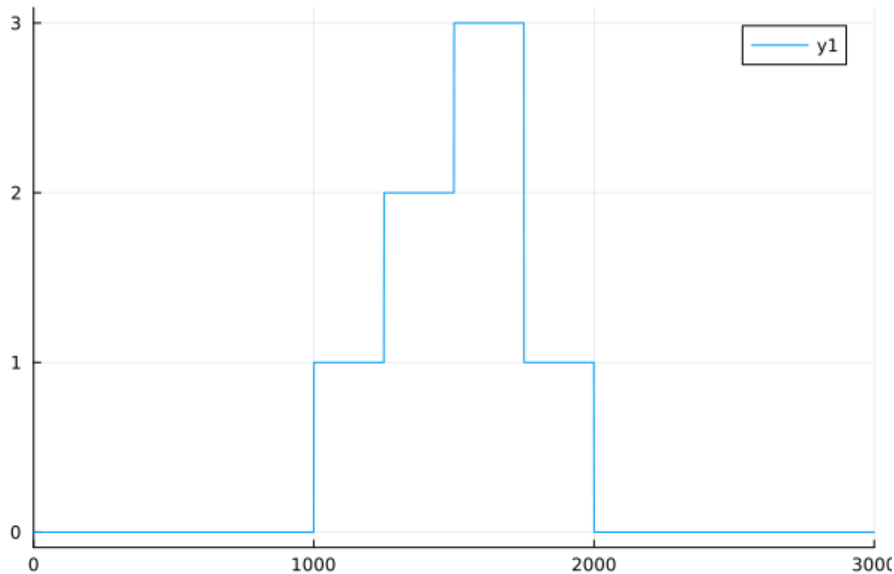
Required:

- modification of all the beam classes to behave this way for DIS, RES events. **DONE**

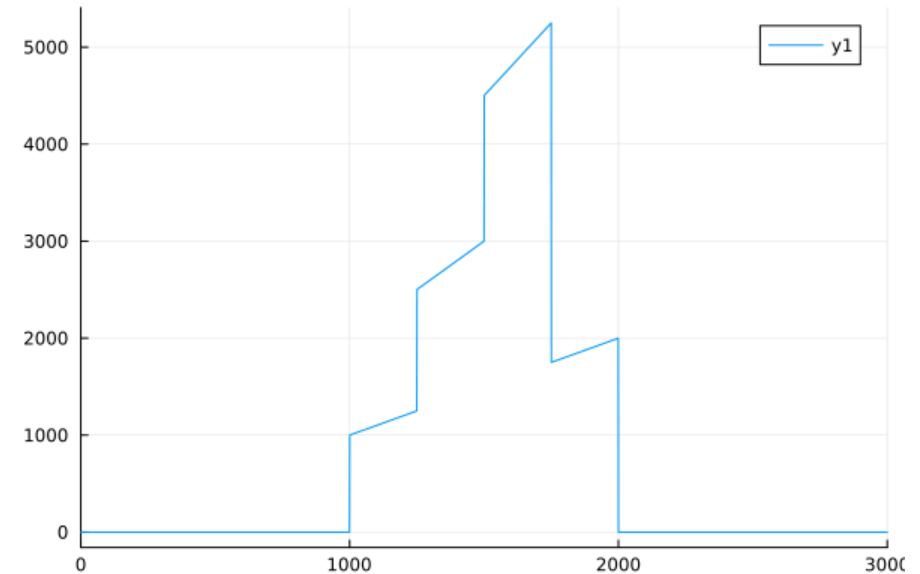
How beam profile needs to change for DIS, RES events

```
beam_type = 0  
beam_energy = 1000 2000 1 2 3 1
```

- in DIS mode every beam energy profile is multiplied by energy



Simple beam profile



Same profile when used in DIS mode.

Forward inclined angular distributions

- For all the dynamics the maximum cross section (before Pauli blocking) occurs in the “forward” direction in the centre-of-mass frame. This effect grows with energy.
- Default algorithm samples the outgoing lepton direction isotropically in the centre-of-mass frame with the weight $= \frac{d\sigma}{d\Omega} \times 4\pi$ which is equivalent to sampling $z = \cos \theta_{\text{CM}}$ uniformly from $[-1, 1]$.
- If we take random $x \in [0, 1]$ and set $z = \cos \theta_{\text{CM}} = 1 - 2x$, then $x = 0$ corresponds to forward direction where $\frac{d\sigma}{d\Omega}$ has peak.
- Instead we can take $z = 1 - 2x^2$ and set weight $= \frac{d\sigma}{d\Omega} \times 4\pi \times 2x$. Then the peak is multiplied by small values of x and is much smaller.
- This trick yields a **several-fold improvement in efficiency** in each dynamical channel. Especially for high energies.

Remarks

- Instead of $z = 1 - x^2$ one can use $z = 1 - x^n$ for any $n > 1$ (e.g. $n = 1.5$).
- It is also possible to make n depend on neutrino energy and dynamical channel.
- However, $n = 2$ gives robust results and used is most often.

Problems with efficiency in Coherent channel

- Efficiencies of order 10^{-5} were common in coherent channel. Efficiency dynamically decreased during event production decreased when new cross section maxima appeared and production of events could not finish. The same problem also occurred in **GENIE**.
- The expression for the cross section contained the factor $\exp(-T)$ and most of the cross section came from a very small region.
- Elimination of this factor was possible by:
 - the substitution $T = -\ln(x)$, changing the order of integration
 - calculation of the new integration limits and the Jacobian.
- **Result:** an increase in efficiency by **two orders of magnitude**, and, when combined with non-uniform sampling of q_0 , by **three orders of magnitude**.
- Problem solved.

FSI run only for real events

- The cascade code is performed only for real events, and only after they have already been accepted. So it is (almost) never done in vain.

Separate real event production for every channels

- When using separate event production for each channel:

$$\text{Efficiency}_{channel} = \frac{\text{avg weight}_{channel}}{\text{max weight}_{channel}} \gg \frac{\text{avg weight}_{channel}}{\text{max weight}_{all\ channels}}$$

- This is possible because the **channel cross sections** and **maximum weights** are obtained during the test run. With this **numbers of real events** needed for every channel are easily obtained.

Lazy purged events elimination

- **maximum_weights** in each channel can increase during real event production.
- When it happens some of the already accepted n_{prev} events need to be purged from file so that only n_{new} events remain, where:

$$n_{new} = n_{prev} \frac{\max \text{ weight}_{prev}}{\max \text{ weight}_{new}}$$

Lazy approach:

- This may happen many times without really deleting the purged events from the file.
- Purged events are assumed to reside at the beginning of the file.
- Events for each channel are stored in different files.
- Purged events are omitted, when copying all channels to one file after simulations are completed.

Thank You